What is Latent Semantic Analysis (LSA)?

Latent Semantic Analysis (LSA) is a **statistical technique** used in **Natural Language Processing** to:

discover **hidden (latent) semantic relationships** between words and documents by analyzing patterns of word usage.

One-line definition (Exam-ready):

LSA is a dimensionality-reduction technique that applies Singular Value Decomposition (SVD) on a term–document matrix to capture latent semantic structure.

Workflow of LSA

Text Documents

↓

Term–Document Matrix (TDM)

↓

Singular Value Decomposition (SVD)

↓

Reduced Semantic Space

↓

Semantic similarity between words & documents

Mathematical Foundation (IMPORTANT)

LSA is based on **Singular Value Decomposition (SVD)**.

If **A** is the Term–Document Matrix:

$$A = U\Sigma V^T$$

Where:

- **U** → Term–Concept matrix
- **Σ (Sigma)** → Singular values (importance of concepts)

- $\mathbf{V^T}$ → Concept–Document matrix

Dimensionality Reduction:

We keep only **top k singular values**:

$$A_k = U_k \Sigma_k V_k^T$$

This creates a **latent semantic space**

Example:

Step 1: Documents

D1: Yoga improves mental health

D2: Meditation improves emotional health

D3: Yoga and meditation are beneficial

Step 2: Term–Document Matrix (TDM)

Term	**D1**	**D2**	**D3**
yoga	1	0	1
meditation	0	1	1
improves	1	1	0
mental	1	0	0
emotional	0	1	0
health	1	1	0
beneficial	0	0	1

This matrix **A** is sparse and high-dimensional

Step 3: Apply SVD

$$A = U \Sigma V^T$$

After SVD:

- Concepts discovered automatically
- Example latent concepts:
    - **Concept 1 → Health & Well-being**
    - **Concept 2 → Yoga & Meditation practices**

Concepts are **not explicitly labelled** — we interpret them.

Step 4: Reduce Dimensions (k = 2)

We keep only top **2 singular values**:

$$A_2 = U_2 \Sigma_2 V_2^T$$

Now:

- Noise removed
- Semantically related words move **closer**
- "Yoga" & "Meditation" become related **even if not always co-occurring**

Intuitive Explanation

Think of LSA like:

Reading many documents and **understanding themes**, instead of counting exact words.

- Humans understand meaning
- LSA **approximates human understanding** using linear algebra

Python Implementation (For Understanding)

```
from sklearn.feature_extraction.text import CountVectorizer

from sklearn.decomposition import TruncatedSVD

import pandas as pd
```

```
documents = [
    "Yoga improves mental health",
    "Meditation improves emotional health",
    "Yoga and meditation are beneficial"
]
```

Step 1: Create TDM

```
vectorizer = CountVectorizer(stop_words='english')
X = vectorizer.fit_transform(documents)
pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
```

Step 2: Apply LSA (SVD)

```
lsa = TruncatedSVD(n_components=2)
X_lsa = lsa.fit_transform(X)
```

Reduced Semantic Space

```
pd.DataFrame(X_lsa, columns=['Concept1', 'Concept2'])
```

Explained Variance

```
print(lsa.explained_variance_ratio_)
```

Interpretation of Results

- Documents with similar meaning have **closer vectors**
- Semantic similarity can be computed using **cosine similarity**
- LSA captures **latent themes**, not surface words

Advantages of LSA

✓ Handles synonymy
✓ Reduces dimensionality
✓ Noise reduction
✓ Works without labeled data

Limitations of LSA

Linear model
Hard to interpret latent dimensions
Computationally expensive
Cannot fully handle polysemy
Outperformed by Word2Vec, BERT